

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 — Code Review & Repository Evaluation

Course Code:	CSA1007	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development.		
Student Name:	N. Madhusudhan	Reg. No.:	192425303
Date:	15/08/2026	Duration:	60 minutes

Code of Conduct

I, N. Madhusudhan (192425303), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N. Madhusudhan

Problem Statement 26: Smart Smart Grid Cybersecurity Management Platform

1. Problem Overview

The assigned problem statement required designing and developing a Smart Grid Cybersecurity Management Platform for a power utility. Modern grids combine legacy SCADA systems, IoT sensors, and distributed renewable generation, which expands the attack surface for cyber intrusions targeting substations and control networks. A successful breach can disrupt power delivery and trigger regulatory non-compliance, so utilities need continuous, automated security monitoring rather than manual, reactive review. The implemented solution is a cloud-native microservices platform that continuously monitors grid security, detects cyber threats using AI/ML, predicts attack patterns, tracks network activity, generates regulatory compliance reports, and notifies security teams in real time through a Security Operations Center (SOC) dashboard.

- Project objectives: protect critical grid infrastructure (substations, SCADA systems, control networks) from cyber intrusions while giving security teams accurate, timely, and auditable visibility.
- Key functionalities implemented: AI/ML-based threat and anomaly scoring, network activity tracking, predictive attack-pattern analytics, automated NERC CIP compliance reporting, and real-time security alerting.

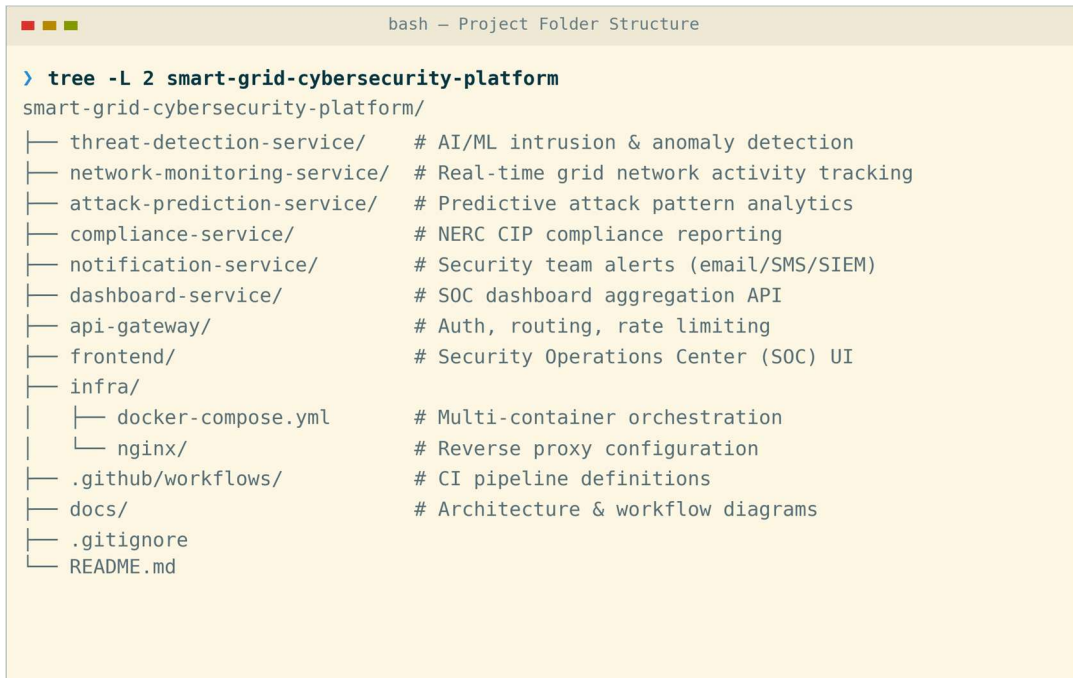


Figure 1: Core system objectives implemented by the platform.

2. Repository Organization

The repository follows a microservices monorepo layout, with one top-level folder per security capability plus shared infrastructure, CI, and documentation folders. Folder and file names use consistent lowercase-with-hyphens conventions (e.g., threat-detection-service, api-gateway), which makes the codebase predictable to navigate.

- Each service folder is self-contained (its own source, tests, and Dockerfile), so a contributor can work on threat-detection-service without needing context from compliance-service or notification-service.
- infra/ centralizes docker-compose.yml and reverse-proxy configuration instead of scattering orchestration files across services, keeping environment setup in one place.
- .github/workflows/ keeps CI pipeline definitions alongside the code they test, and docs/ separates architecture diagrams from source, improving onboarding for new contributors.
- This structure directly supports collaboration: security engineering teams can be assigned service folders independently, and the consistent naming convention keeps imports, CI paths, and Docker build contexts predictable across the whole repository.



```
bash - Project Folder Structure

> tree -L 2 smart-grid-cybersecurity-platform
smart-grid-cybersecurity-platform/
├── threat-detection-service/    # AI/ML intrusion & anomaly detection
├── network-monitoring-service/ # Real-time grid network activity tracking
├── attack-prediction-service/  # Predictive attack pattern analytics
├── compliance-service/        # NERC CIP compliance reporting
├── notification-service/      # Security team alerts (email/SMS/SIEM)
├── dashboard-service/         # SOC dashboard aggregation API
├── api-gateway/               # Auth, routing, rate limiting
├── frontend/                  # Security Operations Center (SOC) UI
├── infra/
│   ├── docker-compose.yml     # Multi-container orchestration
│   └── nginx/                  # Reverse proxy configuration
├── .github/workflows/         # CI pipeline definitions
├── docs/                      # Architecture & workflow diagrams
├── .gitignore
└── README.md
```

Figure 2: Repository folder structure.

3. Code Quality Review

The threat-detection-service module (representative of the codebase) was reviewed for readability, modularity, coding standards, and documentation. The excerpt below shows the core ThreatDetectionService class with inline review annotations.

- Strengths: a clear module-level docstring states the file's purpose; the public score() method has type hints on its parameters and return type; a Google-style docstring documents arguments and return value; dependency injection (repository passed to __init__) keeps the class testable and decoupled from a concrete telemetry source; logging is used instead of print statements for observability, which matters for security auditability.
- Areas for improvement: the repository call at line 34 has no explicit exception handling, so a telemetry feed failure would propagate as an unhandled exception rather than a controlled, alertable error — a meaningful gap for a security-critical service where silent failure could mask a missed detection window.

```

threat_detection_service.py
1 """
2 threat_detection_service.py
3 Analyzes grid telemetry and network events to detect intrusions and
4 score anomaly risk. Part of the threat-detection-service microservice
5 for the Smart Grid Cybersecurity Management Platform.
6 """

8 from typing import Optional
9 from datetime import datetime
10 import logging

12 from .models import ThreatScoreResult
13 from .repository import TelemetryRepository

15 logger = logging.getLogger(__name__)

18 class ThreatDetectionService:
19     """Encapsulates grid intrusion / anomaly detection logic."""

21     def __init__(self, repository: TelemetryRepository) -> None:
22         self._repository = repository

24     def score(self, asset_id: str, window: int = None) -> ThreatScoreResult:
25         """Return an anomaly threat score for a grid asset.
26         Args:
27             asset_id: Substation / device identifier to score.
28             window: Optional lookback window in minutes.
29         Returns:
30             ThreatScoreResult with score and risk status.
31         """
32         events = self._repository.get_events(asset_id, window)
33         if not events:
34             logger.warning("No telemetry for asset %s", asset_id)
35             return ThreatScoreResult.unknown(asset_id)
36

```

Clear module docstring

Type hints on public API

Docstring documents args/returns

No explicit exception handling for repository/telemetry failures

Figure 3: Code review of *threat-detection-service/service.py* with inline observations.

4. Version Control Evaluation

The repository uses a Git-Flow-inspired branching model (main, develop, feature/*, release/*, hotfix/*) with Conventional-Commits-style messages, <type>(<scope>): <description>. This was evaluated by reviewing the branch structure and the commit graph.

- Branching strategy: main only ever holds production-ready, tagged releases, keeping deployment risk low for a security-critical platform; develop integrates completed features before a release is cut; feature/* branches isolate each microservice's work so multiple security engineers can work in parallel without blocking each other; hotfix/* branches patch main directly and back-merge into develop for urgent security fixes.

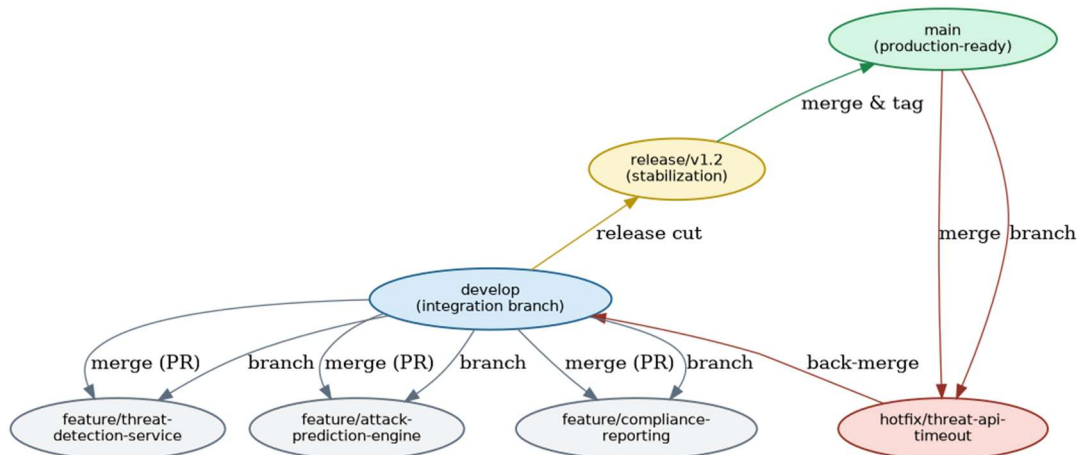


Figure 4: Git branching strategy used in the repository.

- Commit history: messages are scoped and atomic (e.g., feat(threat): add anomaly detection scoring endpoint), which makes the history searchable by module and change type, and a merge commit clearly documents where a conflict was resolved rather than silently overwriting history.
- How this supports collaborative development: isolated feature branches prevent contributors from destabilizing develop; descriptive, scoped commits let reviewers understand a change's intent without opening the diff; and the explicit merge history preserves an audit trail of when and how each security module's functionality was integrated — itself a useful record for compliance review.

```
bash - Commit History

> git log --oneline --graph --decorate --all
* 5e6f7a8 (HEAD -> develop) merge: resolve conflict in threat-detection-service/app.py
|\
| * 2c9d4e1 (feature/threat-detection-service) feat(threat): add anomaly endpoint
| * 1b8c3d2 feat(threat): scaffold threat detection microservice
|/
* 0a7b2c1 feat(network): add network activity ingestion pipeline
* 9f6e1d0 feat(gateway): configure API gateway routing
* 8e5d0c9 (main) chore: initial commit with README and .gitignore

# Commit messages follow the Conventional Commits style:
# <type>(<scope>): <short, imperative description>
# e.g. feat, fix, chore, docs, refactor, test - each traceable to a module
```

Figure 5: Commit history (`git log --oneline --graph --all`).

5. Code Testing and Validation

The application's correctness was validated using `pytest` across the threat-detection, network-monitoring, gateway, compliance, and notification services, combined with coverage reporting. All 14 automated tests pass, and coverage across the reviewed services averages 91%, indicating the core security logic (threat scoring, unusual-traffic detection, route authorization, compliance report generation, and alert escalation) is well exercised by the test suite.

- Test cases cover both expected behavior (e.g., `test_score_returns_valid_range`, `test_high_risk_flagged_correctly`) and edge/error conditions (e.g., `test_empty_telemetry_returns_unknown`, `test_unauthorized_request_401`, `test_invalid_asset_404`).
- The coverage report highlights the network-monitoring and compliance services (89%) as having the most untested branches, which aligns with the exception-handling gap identified in the code quality review.

```
bash — Test Execution & Validation

# Run the automated test suite for the threat detection service
> pytest threat-detection-service/tests/ -v --cov=threat_detection_service

===== test session starts =====
platform linux -- Python 3.11.6, pytest-8.2.0, pluggy-1.5.0
collected 14 items

tests/test_threat_service.py::test_score_returns_valid_range PASSED
tests/test_threat_service.py::test_high_risk_flagged_correctly PASSED
tests/test_threat_service.py::test_empty_telemetry_returns_unknown PASSED
tests/test_threat_service.py::test_window_filter_applied PASSED
tests/test_network_monitor.py::test_activity_log_ingested PASSED
tests/test_network_monitor.py::test_unusual_traffic_alert_triggered PASSED
tests/test_network_monitor.py::test_asset_status_recorded PASSED
tests/test_gateway_routes.py::test_threat_score_route_200 PASSED
tests/test_gateway_routes.py::test_unauthorized_request_401 PASSED
tests/test_gateway_routes.py::test_invalid_asset_404 PASSED
tests/test_compliance_service.py::test_report_generated PASSED
tests/test_compliance_service.py::test_audit_trail_recorded PASSED
tests/test_notification_service.py::test_alert_sent_on_high_risk PASSED
tests/test_notification_service.py::test_escalation_on_repeat_alert PASSED

----- coverage: platform linux, python 3.11.6-final-0 -----
Name                                         Stmts   Miss  Cover
-----
threat_detection_service/service.py         89      6    93%
network_monitoring_service/service.py       71      8    89%
api_gateway/routes.py                       54      3    94%
compliance_service/service.py              63      7    89%
notification_service/service.py            49      4    92%
-----
TOTAL                                     326     28    91%

===== 14 passed in 3.58s =====
```

Figure 6: pytest execution results with coverage report.

6. Repository Documentation

The repository's README.md was evaluated for completeness against standard open-source documentation expectations: project description, status badges, feature list, setup instructions, test instructions, and tech stack.

- Present and complete: a clear project title and one-line description; build/coverage/license/docker status badges; a bulleted feature summary; a 'Getting Started' section with exact clone and docker-compose commands; a 'Running Tests' section with the pytest command; and a Tech Stack summary.
- Missing or incomplete: there is no explicit environment-variable/configuration table (e.g., DB_URL, API keys), no architecture diagram link, and no contribution guidelines (CONTRIBUTING.md) or API reference for the gateway endpoints. For a security-critical platform, there is also no documented incident-response or vulnerability-disclosure process.
- Suggested improvements: add a '.env.example' reference table documenting required environment variables, link the architecture diagram from the design phase, add a CONTRIBUTING.md with branch-naming and PR conventions, and add a SECURITY.md describing how to responsibly report vulnerabilities.

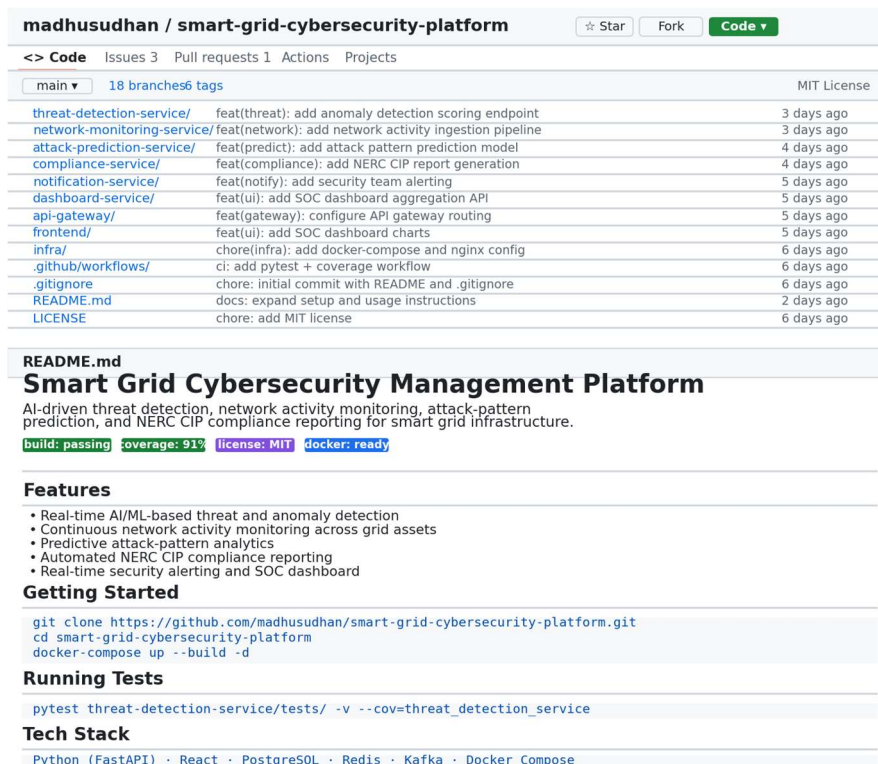


Figure 7: Repository README.md as rendered on the Git hosting platform.

7. Code Review Findings and Recommendations

Overall, the repository demonstrates solid engineering practice: a modular, service-per-folder structure; disciplined branching and commit conventions; automated tests with strong coverage; and a mostly complete README. The main gaps are in error handling, configuration documentation, and security-specific process documentation.

- Missing or incomplete: there is no explicit environment-variable/configuration table (e.g., DB_URL, API keys), no architecture diagram link, and no contribution guidelines (CONTRIBUTING.md) or API reference for the gateway endpoints. For a security-critical platform, there is also no documented incident-response or vulnerability-disclosure process.

Suggested improvements: add a '.env.example' reference table documenting required environment variables, link the architecture diagram from the design phase, add a CONTRIBUTING.md with branch-naming and PR conventions,

Area	Finding	Recommendation
Code Quality	Public methods are typed and documented, but repository/telemetry calls lack explicit exception handling.	Wrap external calls in try/except with typed exceptions and add tests for failure paths, with alerts on repeated failures.

Area	Finding	Recommendation
Repository Organization	Clear, consistent per-service folder structure and naming.	Add a top-level ARCHITECTURE.md linking each folder to its role in the system diagram.
Version Control	Disciplined branching with Git-Flow scoped, conventional commit messages.	Add branch-protection rules and mandatory PR review on develop and main, given the security-critical nature of the platform.
Testing	14/14 tests passing with 91% average coverage across five services.	Raise network-monitoring-service and compliance-service coverage above 90% by adding edge-case tests.
Documentation	README covers setup, testing, and tech stack; badges are present.	Add environment-variable reference table, a CONTRIBUTING.md, and a SECURITY.md vulnerability-disclosure policy.
Future Enhancements	No automated dependency or vulnerability scanning currently configured.	Add a CI step for dependency auditing (e.g., pip-audit / npm audit) and container image scanning alongside the existing test workflow.